



Backpropagation Optimizers

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes

১৪. Backpropagation and Optimizers 🗨️ ⚙️

— নিউরাল নেটওয়ার্ক কীভাবে শেখে, Chain Rule ও SGD/Adam Optimizer

সিরিজ: Machine Learning & Deep Learning Notes | পর্ব ২ — ডিপ লার্নিং

পূর্ববর্তী নোট: ANN Introduction

পরবর্তী নোট: Activation Functions (শীঘ্রই আসছে)

১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)

এই concept কী এবং কেন দরকার?

কল্পনা করো তুমি একটি তীরন্দাজ — লক্ষ্যে তীর ছুঁড়ছ। প্রথম তীর ছুঁড়লে লক্ষ্য থেকে ২ হাত ডানে গেছে। এখন তুমি কী করবে? একটু বামে তাক করবে। আবার ছুঁড়লে দেখলে এবার একটু উপরে গেছে। তাহলে একটু নিচে তাক করলে।

এভাবে বারবার ভুল থেকে শিখে শিখে তুমি পারফেক্ট হয়ে উঠছ। **Backpropagation ঠিক এভাবেই কাজ করে।**

নিউরাল নেটওয়ার্ক প্রথম prediction দেয় — সেটি ভুল হয়। তারপর সেই ভুল (error) পেছন দিকে পাঠিয়ে দেয় এবং বলে, "কার কার জন্য এই ভুল হলো?" প্রতিটি weight তার দোষ স্বীকার করে এবং নিজেকে একটু ঠিক করে নেয়। এই প্রক্রিয়া বারবার চলে — প্রতিবার model আরও ভালো হয়।

বাস্তব উদাহরণ — রান্না শেখা:

- **ভুল মাপা** = Loss Calculation
- **ভুলের কারণ খোঁজা** = Backpropagation
- **নিজেকে ঠিক করা** = Weight Update (Optimizer)

এটি কোন সমস্যা সমাধান করে?

আগে আমরা শিখেছিলাম — ANN-এ Forward Propagation করে prediction দেওয়া যায়। কিন্তু সমস্যা হলো: **নেটওয়ার্ক কীভাবে জানবে কোথায় ভুল হলো এবং weight কীভাবে পরিবর্তন করতে হবে?**

Backpropagation সেই উত্তর দেয়। এটি একটি algorithm যা:

1. Error পেছন দিকে পাঠায়
2. প্রতিটি weight কতটুকু দায়ী সেটি গণনা করে
3. Optimizer সেই তথ্য ব্যবহার করে weight update করে

২. বিস্তারিত ব্যাখ্যা (Deep Explanation)

সম্পূর্ণ Training Loop — ৪টি বড় ধাপ:

[১] Forward Pass → [২] Loss Calculation → [৩] Backward Pass → [৪] Weight Update
↑
(এই cycle বারবার চলে)

ধাপ ১: Forward Pass (সামনের দিকে যাওয়া)

Input data নেটওয়ার্কের মধ্যে দিয়ে সামনে যায় এবং output তৈরি হয়।

Input (X) → Layer 1 → Layer 2 → Output ($\hat{y}$)

প্রতিটি layer এই কাজ করে:

- **Weighted Sum:** $z = W*a + b$
- **Activation:** $a = \text{sigmoid}(z)$

ধাপ ২: Loss Calculation (ভুল মাপা)

Loss function দিয়ে বোঝা যায় আমাদের prediction কতটা ভুল।

উদাহরণ: Mean Squared Error (MSE) = (actual - predicted) squared

ধাপ ৩: Backward Pass — Backpropagation

এটিই মূল কাজ। আমরা জানতে চাই প্রতিটি weight পরিবর্তন করলে Loss কতটুকু পরিবর্তন হবে।

এটিকে গণিতে বলে: dL/dw (L-এর w-এর সাপেক্ষে partial derivative)।

কিন্তু সমস্যা হলো — Loss থেকে সরাসরি একটি weight পর্যন্ত অনেক layer আছে। এখানেই **Chain Rule** ব্যবহার হয়।

Chain Rule — সহজ ভাষায়:

পরিস্থিতি কল্পনা করো:

তুমি (x) → তোমার বেতন (y) → তোমার সঞ্চয় (z)

- তুমি ১ ঘণ্টা বেশি কাজ করলে বেতন ৫০০ টাকা বাড়ে
- বেতন ১০০ টাকা বাড়লে সঞ্চয় ৮০ টাকা বাড়ে
- তাহলে, ১ ঘণ্টা বেশি কাজ করলে সঞ্চয় কত বাড়বে?

$$dz/dx = dz/dy * dy/dx = (80/100) * (500/1) = 400 \text{ টাকা}$$

এটাই Chain Rule! একটি শৃঙ্খলের মাঝখান দিয়ে derivative বের করা।

Neural Network-এ Chain Rule:

একটি layer-এর weight (w)-এর জন্য:

$$dL/dw = (dL/da) * (da/dz) * (dz/dw)$$

- **dL/da** : Loss থেকে activation পর্যন্ত gradient
- **da/dz** : Activation function-এর derivative
- **dz/dw** : Weighted sum থেকে weight পর্যন্ত (= পূর্ববর্তী layer-এর activation)

ধাপ ৪: Weight Update — Optimizer

Gradient পাওয়ার পরে Optimizer এই formula দিয়ে weight update করে:

$$w_{\text{new}} = w_{\text{old}} - \alpha * (dL/dw)$$

এখানে **alpha** হলো **Learning Rate** — কতটুকু পদক্ষেপ নেবে।

৩. Math / Theory

৩.১ একটি Simple Network-এ পুরো Calculation:

Network Structure:

$$x \rightarrow z1 = w1*x + b1 \rightarrow a1 = \text{sigmoid}(z1) \rightarrow z2 = w2*a1 + b2 \rightarrow y_{\text{hat}}$$

Loss (MSE): $L = (y - y_{\text{hat}})^2$

Backpropagation — পেছন থেকে শুরু:

Step 1 — Loss-এর output সাপেক্ষে gradient:

$$dL/dy_{\text{hat}} = -2(y - y_{\text{hat}})$$

Step 2 — Output layer-এর weight $w2$ সাপেক্ষে:

$$\begin{aligned} dL/dw2 &= dL/dy_{\text{hat}} * dy_{\text{hat}}/dw2 \\ &= -2(y - y_{\text{hat}}) * a1 \end{aligned}$$

Step 3 — Hidden layer-এর weight $w1$ সাপেক্ষে (Chain Rule ব্যবহার):

$$\begin{aligned} dL/dw1 &= dL/dy_{\text{hat}} * dy_{\text{hat}}/da1 * da1/dz1 * dz1/dw1 \\ &= -2(y - y_{\text{hat}}) * w2 * \text{sigmoid_prime}(z1) * x \end{aligned}$$

যেখানে **sigmoid_prime(z)** হলো Sigmoid function-এর derivative:

```
sigmoid_prime(z) = sigmoid(z) * (1 - sigmoid(z))
```

৩.২ Manual Numerical Example:

ধরি:

- $x = 2.0$, y (actual) = 1.0
- $w1 = 0.5$, $b1 = 0.1$, $w2 = 0.8$, $b2 = 0.2$
- Learning rate $\alpha = 0.1$

Forward Pass:

```
z1 = 0.5 * 2 + 0.1 = 1.1  
a1 = sigmoid(1.1) = 1/(1+e^-1.1) = 0.7503  
z2 = 0.8 * 0.7503 + 0.2 = 0.8002  
y_hat = 0.8002  
  
Loss = (1.0 - 0.8002)^2 = (0.1998)^2 = 0.03992
```

Backward Pass:

```
dL/dy_hat = -2(1.0 - 0.8002) = -0.3996  
  
dL/dw2 = -0.3996 * 0.7503 = -0.2999  
  
sigmoid_prime(z1) = 0.7503 * (1 - 0.7503) = 0.1875  
  
dL/dw1 = -0.3996 * 0.8 * 0.1875 * 2 = -0.1199
```

Weight Update:

```
w2_new = 0.8 - 0.1 * (-0.2999) = 0.8 + 0.02999 = 0.8300  
w1_new = 0.5 - 0.1 * (-0.1199) = 0.5 + 0.01199 = 0.5120
```

Loss কমেছে! Weight দুটো আরও ভালো হয়েছে।

৩.৩ SGD (Stochastic Gradient Descent) Formula:

Basic SGD:

$$\theta(t+1) = \theta(t) - \eta * \text{gradient}_J(\theta(t))$$

- **θ** = Parameters (weights and biases)
- **η** = Learning rate (সাধারণত 0.01 বা 0.001)
- **gradient_J** = Gradient of loss function

SGD with Momentum:

$$\begin{aligned} v(t) &= \beta * v(t-1) + (1-\beta) * \text{gradient}_J(\theta(t)) \\ \theta(t+1) &= \theta(t) - \eta * v(t) \end{aligned}$$

- **$v(t)$** = Velocity (momentum term)
- **β** = Momentum coefficient (সাধারণত 0.9)

৩.৪ Adam Optimizer Formula:

Adam = Adaptive Moment Estimation

Step 1 — First Moment (Momentum):

$$m(t) = \beta_1 * m(t-1) + (1 - \beta_1) * g(t)$$

Step 2 — Second Moment (RMSProp-like):

$$v(t) = \beta_2 * v(t-1) + (1 - \beta_2) * g(t)^2$$

Step 3 — Bias Correction:

$$m_hat(t) = m(t) / (1 - beta1^t)$$
$$v_hat(t) = v(t) / (1 - beta2^t)$$

Step 4 — Weight Update:

$$theta(t+1) = theta(t) - eta * m_hat(t) / (sqrt(v_hat(t)) + epsilon)$$

Symbol	অর্থ	Default মান
$g(t)$	Current gradient	—
$m(t)$	First moment (mean of gradients)	0
$v(t)$	Second moment (variance of gradients)	0
$beta1$	First moment decay rate	0.9
$beta2$	Second moment decay rate	0.999
eta	Learning rate	0.001
$epsilon$	Zero division ঠেকাতে	1e-8

8. Code Example (Python)

```

import numpy as np

# ---- Activation Functions ----
def sigmoid(z):
    # overflow ঠেকাতে clip ব্যবহার
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def sigmoid_derivative(z):
    # sigmoid-এর derivative: s*(1-s)
    s = sigmoid(z)
    return s * (1 - s)

def mse_loss(y_true, y_pred):
    # Mean Squared Error Loss
    return np.mean((y_true - y_pred) ** 2)

# -----
# Simple Neural Net with Backpropagation + SGD
# -----
class SimpleNeuralNet:
    def __init__(self, learning_rate=0.1):
        np.random.seed(42)
        # 2 inputs, 3 hidden neurons, 1 output
        self.W1 = np.random.randn(2, 3) * 0.1
        self.b1 = np.zeros((1, 3))
        self.W2 = np.random.randn(3, 1) * 0.1
        self.b2 = np.zeros((1, 1))
        self.lr = learning_rate

    def forward(self, X):
        # Hidden Layer: weighted sum + activation
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = sigmoid(self.z1)
        # Output Layer
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = sigmoid(self.z2)
        return self.a2

    def backward(self, X, y):
        m = X.shape[0]
        # Output layer gradient (Chain Rule)
        dL_da2 = -2 * (y - self.a2) / m

```



```

dL_dz2 = dL_da2 * sigmoid_derivative(self.z2)
dL_dw2 = np.dot(self.a1.T, dL_dz2)
dL_db2 = np.sum(dL_dz2, axis=0, keepdims=True)

# Hidden layer gradient (আরেকটি Chain Rule step)
dL_da1 = np.dot(dL_dz2, self.W2.T)
dL_dz1 = dL_da1 * sigmoid_derivative(self.z1)
dL_dw1 = np.dot(X.T, dL_dz1)
dL_db1 = np.sum(dL_dz1, axis=0, keepdims=True)

# SGD Update: w = w - lr * gradient
self.W2 -= self.lr * dL_dw2
self.b2 -= self.lr * dL_db2
self.W1 -= self.lr * dL_dw1
self.b1 -= self.lr * dL_db1

def train(self, X, y, epochs=1000):
    losses = []
    for epoch in range(epochs):
        y_pred = self.forward(X)
        loss = mse_loss(y, y_pred)
        losses.append(loss)
        self.backward(X, y)
        if epoch % 200 == 0:
            print(f"Epoch {epoch:4d} | Loss: {loss:.6f}")
    return losses

# -----
# Adam Optimizer – শূন্য থেকে Implementation
# -----

class AdamOptimizer:
    def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.eps = eps
        self.t = 0
        self.m = {}
        self.v = {}

    def update(self, params, grads):
        self.t += 1
        updated = {}

```

```

    for key in params:
        if key not in self.m:
            self.m[key] = np.zeros_like(params[key])
            self.v[key] = np.zeros_like(params[key])
        # First moment (momentum)
        self.m[key] = self.beta1*self.m[key] + (1-self.beta1)*grads[key]
        # Second moment (adaptive LR)
        self.v[key] = self.beta2*self.v[key] + (1-self.beta2)*grads[key]**2
        # Bias correction
        m_hat = self.m[key] / (1 - self.beta1**self.t)
        v_hat = self.v[key] / (1 - self.beta2**self.t)
        # Parameter update
        updated[key] = params[key] - self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
    return updated

# -----
# Test - XOR Problem (non-linearly separable)
# -----
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

print("=" * 50)
print("    Backpropagation দিয়ে XOR শেখা (SGD)")
print("=" * 50)

model = SimpleNeuralNet(learning_rate=0.5)
model.train(X, y, epochs=1000)

print("\nFinal Predictions:")
preds = model.forward(X)
for i in range(4):
    print(f"  Input: {X[i]} -> {preds[i][0]:.4f} | Actual: {y[i][0]}")

acc = np.mean((preds > 0.5).astype(int) == y) * 100
print(f"\n  Accuracy: {acc:.1f}%")

```

Expected Output:

Backpropagation দিয়ে XOR শেখা (SGD)

```
Epoch 0 | Loss: 0.249971
Epoch 200 | Loss: 0.249812
Epoch 400 | Loss: 0.241730
Epoch 600 | Loss: 0.082341
Epoch 800 | Loss: 0.019823
```

Final Predictions:

```
Input: [0 0] -> 0.0782 | Actual: 0
Input: [0 1] -> 0.9143 | Actual: 1
Input: [1 0] -> 0.9142 | Actual: 1
Input: [1 1] -> 0.1098 | Actual: 0
```

Accuracy: 100.0%

🔗 🎨 Visual / Diagram

🔗.১ Backpropagation Flow:

=== FORWARD PASS (সামনে) ===

```
Input      Hidden Layer      Output      Loss
[x1] --W1--> [z=Wx+b] --W2--> [y_hat] --> [L]
[x2]          [a=sig(z)]
```

=== BACKWARD PASS (পেছনে) ===

```
dL/dw1 <-- dL/da1 <-- dL/da2 <-- dL/dy_hat
(Chain Rule দিয়ে gradient layer-by-layer পেছনে আসে)
```

=== WEIGHT UPDATE ===

```
SGD: w = w - alpha * dL/dw
Adam: w = w - alpha * m_hat / (sqrt(v_hat) + eps)
```

🔗.২ SGD vs Adam — Loss Journey:

```

Loss
|
| * <- SGD শুরু
| \
|  \__ (SGD ধীরে নামে)
|      \__
|          \__ * <- SGD শেষ
|
| * <- Adam শুরু
| \ (Adam দ্রুত নামে)
|  \__ * <- Adam শেষ (দ্রুত!)
|
+-----> Epochs

```

৫.৩ Adam-এর ভেতরের কাজ:

```

Gradient g(t)
|
+-----+
v               v

First Moment m(t)   Second Moment v(t)
= beta1*m + (1-b1)*g = beta2*v + (1-b2)*g^2
[Momentum: দিক]      [Adaptive LR: গতি নিয়ন্ত্রণ]
|                      |
v Bias Correct        v Bias Correct
m_hat                 v_hat
|                      |
+-----+-----+
v
Update = eta * m_hat / (sqrt(v_hat) + eps)
v
theta_new = theta_old - Update

```

৫.৪ Chain Rule — ধাপে ধাপে:

```

L --> y_hat --> z2 --> a1 --> z1 --> w1
      |
dL/dw1 = dL/dy_hat * dy_hat/dz2 * dz2/da1 * da1/dz1 * dz1/dw1
        = -2(y-y_hat) * 1 * w2 * sigmoid'(z1) * x
(সব কিছু শৃঙ্খলের মতো গুণ করো!)

```

৬. Real-world Use Cases

১. ChatGPT / GPT-4 (OpenAI):

GPT মডেল ট্রেন করতে Adam optimizer ব্যবহার করা হয়। Billions of parameters backpropagation দিয়ে update করা হয়।

২. Google Photos — Image Recognition:

ফটো থেকে মানুষের মুখ, জায়গা চেনার জন্য CNN ট্রেন করা হয় backpropagation দিয়ে।

৩. Tesla — Self Driving Cars:

গাড়ির camera থেকে object detection করার neural network Adam optimizer দিয়ে ট্রেন হয়।

৪. Spotify — Music Recommendation:

তোমার পছন্দের গান থেকে recommendation model ট্রেন করতে SGD ব্যবহার হয়।

৫. AlphaFold (DeepMind — Google):

Protein structure prediction-এর বিশাল model backpropagation + Adam দিয়ে ট্রেন হয় এবং চিকিৎসা বিজ্ঞানে বিপ্লব এনেছে।

৭. Pros & Cons

SGD:

সুবিধা	অসুবিধা
সরল এবং বোঝা সহজ	Learning rate manually tune করতে হয়
Memory কম লাগে	Slow convergence
Generalization ভালো (flat minima খোঁজে)	Noisy gradients-এ oscillate করে
Saddle point থেকে বের হতে পারে	সব parameter-এ এক learning rate
Large dataset-এ কার্যকর	Momentum ছাড়া খুব ধীর

Adam Optimizer:

সুবিধা	অসুবিধা
প্রতিটি parameter-এর জন্য adaptive LR	Memory বেশি লাগে (m, v store)
Default settings-ই কাজ করে	কখনো sharp minima-তে আটকে যায়
Sparse gradient-এ দুর্দান্ত	কখনো SGD-এর চেয়ে generalize কম
দ্রুত convergence	Weight decay সঠিকভাবে না করলে সমস্যা
NLP এবং Computer Vision-এ সেরা	একটু complex implementation

SGD vs Adam — তুলনামূলক চার্ট:

বৈশিষ্ট্য	SGD	Adam
Learning Rate	Fixed	Adaptive (per-parameter)
Momentum	Optional	Built-in
Convergence	ধীর	দ্রুত
Tuning	Sensitive	Robust
Generalization	প্রায়ই ভালো	কখনো একটু কম

৮. ⚠️ Common Mistakes & Gotchas

ভুল ১: Learning Rate অনেক বড় দেওয়া

```
# ভুল – Loss explode করবে
model = SimpleNeuralNet(learning_rate=10.0)

# সঠিক – ছোট learning rate দিয়ে শুরু করো
model = SimpleNeuralNet(learning_rate=0.01)
```

ভুল ২: Gradient Check না করা

যদি loss কমছে না বা উল্টো বাড়ছে — gradient calculation ভুল হতে পারে।

ভুল ৩: Weight Initialization শূন্য করা

```
# ভুল – সব weight শূন্য থাকলে সব neuron একই শিখবে (Symmetry Problem)
W = np.zeros((3, 4))

# সঠিক – Random initialization
W = np.random.randn(3, 4) * 0.01
```

ভুল ৪: Vanishing Gradient সম্পর্কে সচেতন না থাকা

Deep network-এ sigmoid activation ব্যবহার করলে gradient খুব ছোট হয়ে যায়।

শুরুর layer-গুলো কার্যত শেখে না। সমাধান: ReLU ব্যবহার করো।

ভুল ৫: Adam-এ Weight Decay না দেওয়া

```
# সাধারণ Adam (Overfitting হতে পারে)
# optimizer = Adam(lr=0.001)

# AdamW – Weight decay সহ (বেশিরভাগ ক্ষেত্রে ভালো)
# optimizer = AdamW(lr=0.001, weight_decay=0.01)
```

ভুল ৬: Gradient Exploding

RNN বা Deep network-এ gradient অনেক বড় হয়ে যেতে পারে।

সমাধান: Gradient Clipping ব্যবহার করো।

৯. 📌 Related Topics

আগে যা জানা দরকার ছিল:

- ANN Introduction — Neurons, layers, forward propagation
- Gradient Descent — Optimization intuition
- Calculus Basics — Derivative, partial derivative
- Activation Functions — Sigmoid, ReLU

পরে যা শেখা উচিত:

- **Activation Functions (Deep Dive)** — ReLU, Tanh, Softmax এর backprop
- **Vanishing/Exploding Gradients** — কেন হয়, কীভাবে সমাধান করতে হয়
- **Batch Normalization** — Training stabilize করার কৌশল
- **Regularization (Dropout, L1/L2)** — Overfitting ঠেকানো
- **CNN Backpropagation** — Image-এ কীভাবে কাজ করে
- **PyTorch Autograd** — Framework-এ automatic differentiation

১০. 🧠 Memory Tricks

মনে রাখার সহজ কৌশল:

Backpropagation = "দোষ ভাগ করা"

ভুল হলো — পেছনে যাও — প্রত্যেকের দোষ বের করো — সবাইকে একটু শুধরে দাও — আবার চেষ্টা করো।

Chain Rule = "ঘুরিয়ে বলা"

বেতনের কারণে সঞ্চয় বাড়লে, আর কাজের কারণে বেতন বাড়লে — সরাসরি বলো কাজের কারণে সঞ্চয় কত বাড়লো।

SGD vs Adam মনে রাখতে:

- SGD = পুরনো মানচিত্র — সহজ, কিন্তু ধীর
- Adam = Smart GPS — নিজে adapt করে, দ্রুত পৌঁছায়

দ্রুত রেফারেন্স কার্ড:

```
Forward Pass:  y_hat = sigma(Wx + b)
Loss:          L = (y - y_hat)^2
Chain Rule:    dL/dw = dL/dy_hat * dy_hat/dz * dz/dw

SGD Update:    w = w - alpha * dL/dw
Adam Update:   w = w - alpha * m_hat / (sqrt(v_hat) + eps)

Adam Defaults: lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8
```

এক লাইনে সারসংক্ষেপ:

"Backpropagation হলো Chain Rule ব্যবহার করে প্রতিটি weight-এর gradient বের করা, এবং Optimizer (SGD বা Adam) সেই gradient দিয়ে weight আপডেট করে নেটওয়ার্ককে ধীরে ধীরে স্মার্ট বানায়।"

তৈরির তারিখ: ২০২৬-০৪-০৪ | সিরিজ: ML & DL Notes (পর্ব ১৪)

 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

 AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes